Connecting Xdebug to PhpStorm

Connecting the Xdebug debugger to PhpStorm should not be a matter of luck. This article tackles the secrets of a simple Xdebug and PhpStorm configuration that just works. However, it does not cover how to debug an application but rather the detailed process of connecting Xdebug to PhpStorm in every useful way: locally and remotely, using the host and a Vagrant machine, both from a web server and from the CLI.

The following quote from a Stack Overflow question is a good summary of the motivation for this article:

> Like many, I've spent more hours debugging my IDE's connection to XDebug than I have using XDebug to debug my programs. I've gotten it to work repeatedly, but [...] I haven't been able to localize what causes XDebug to work or fail. What am I doing wrong? How can I better debug my IDE's connection to Xdebug?

To figure out what's *really* needed to get Xdebug connecting to your IDE, this article strips down every configuration to the *bare minimum*. That way, you can exclude a lot of factors that you may thought are needed to get it working – and gain a better understanding of the important pieces.

Installing and Configuring Xdebug

Of course, Xdebug needs to be installed first. Depending on your operating system and PHP setup, the procedure may differs a bit. For instance, on a Mac with Homebrew PHP 5.6 already installed:

```
$ brew update
$ brew install php56-xdebug
```

On Ubuntu with PHP already installed:

```
$ sudo apt-get update
$ sudo apt-get install php5-xdebug
```

To check if the `xdebug` extension is available, display the installed PHP modules and filter for `xdebug`:

```
$ php -m | grep xdebug
xdebug
```

Or use the `--version` switch and look for *Xdebug* in the output:

```
php --version
PHP 5.6.5 (cli) (built: Feb  1 2015 15:54:31)
Copyright (c) 1997-2014 The PHP Group
Zend Engine v2.6.0, Copyright (c) 1998-2014 Zend Technologies
    with Zend OPcache v7.0.4-dev, Copyright (c) 1999-2014, by Zend Technologies
    with Xdebug v2.2.5, Copyright (c) 2002-2014, by Derick Rethans
```

To check the default configuration of Xdebug:

```
$ php -i | grep xdebug
/etc/php5/cli/conf.d/20-xdebug.ini
xdebug
xdebug support => enabled
xdebug.auto_trace => Off => Off
xdebug.cli_color => 0 => 0
...
```

On the first line of the above output, you can also spot the location of the `xdebug.ini` configuration file. Alternatively you may use:

```
$ php --ini
Configuration File (php.ini) Path: /etc/php5/cli
Loaded Configuration File:         /etc/php5/cli/php.ini
Scan for additional .ini files in: /etc/php5/cli/conf.d
Additional .ini files parsed:      /etc/php5/cli/conf.d/05-opcache.ini,
...
/etc/php5/cli/conf.d/20-xdebug.ini
```

Remembering the *minimum* commitment from the introduction, the only Xdebug configuration used for this article is to just load the extension.

```
# xdebug.ini
[xdebug]
zend_extension="/usr/local/opt/php56-xdebug/xdebug.so"

;xdebug.remote_enable=1
;xdebug.idekey = "PHPSTORM"
;xdebug.remote_connect_back = 1
;xdebug.remote_host = localhost
;xdebug.remote_port = 9000
;xdebug.remote_autostart = 1
;xdebug.remote_handler = dbgp
;xdebug.profiler_enable = 1
;xdebug.profiler_output_dir = "/tmp"
```

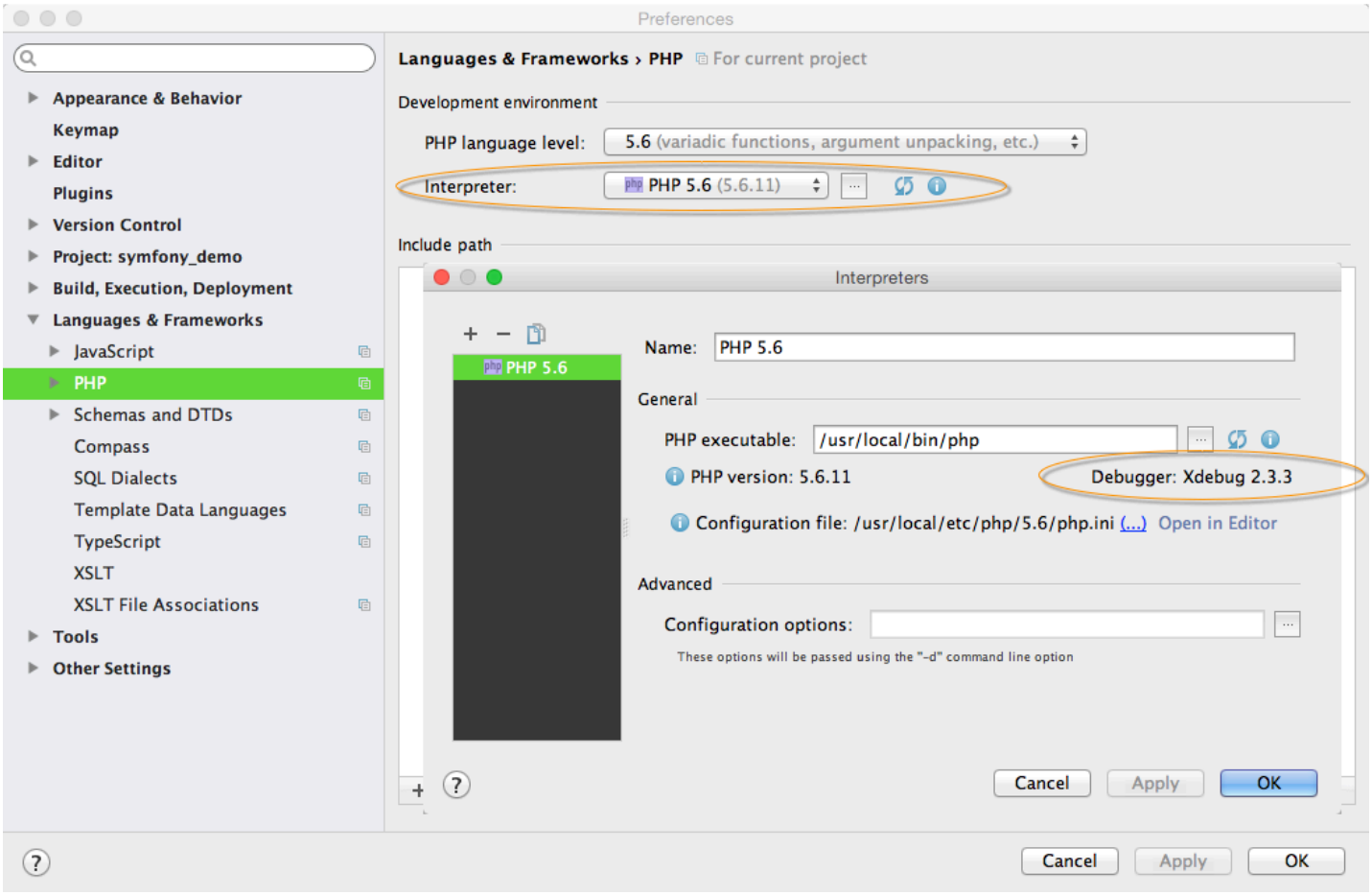
So beside `zend_extension` (poiting to the extension file; value may differs on your system), all other

options are commented out.

Upcoming examples of this articles will be configured in-place. This is done for demo purposes only so there's nothing you have to remember from the Xdebug configuration file. But of course you should persist some configuration in your real environment for convinience.

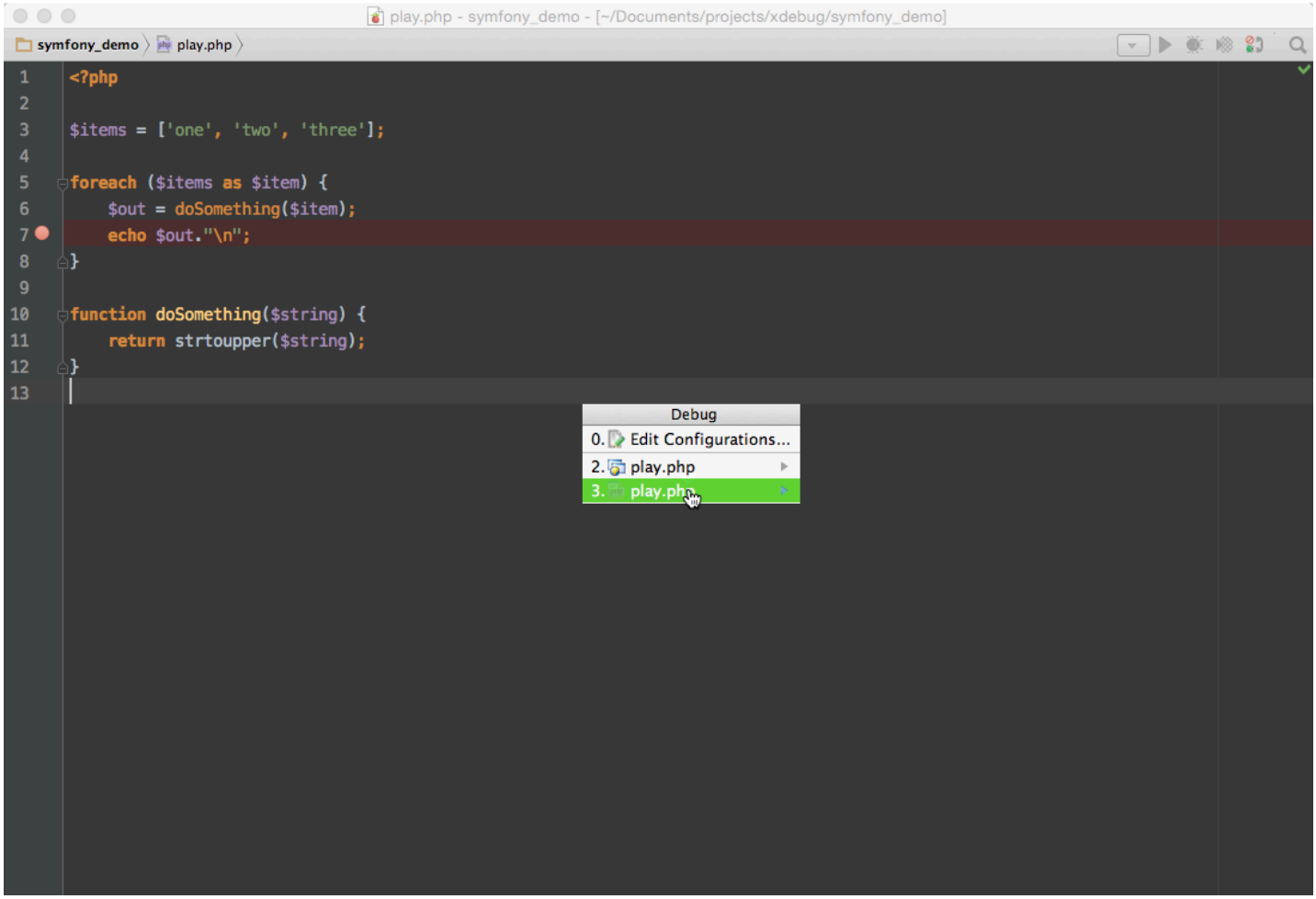
Prepare PhpStorm

Preparing PhpStorm is just a matter of configuring the PHP interpreter. Open the preferences using `Cmd` + `,` and start typing *Interpreter* to find the correct section. In PhpStorm 9 it is in **Languages & Frameworks > PHP**, but this may differ in other versions. Select the PHP interpreter you want to use, click on the icon to the right of the selection and check if Xdebug was properly recognized:

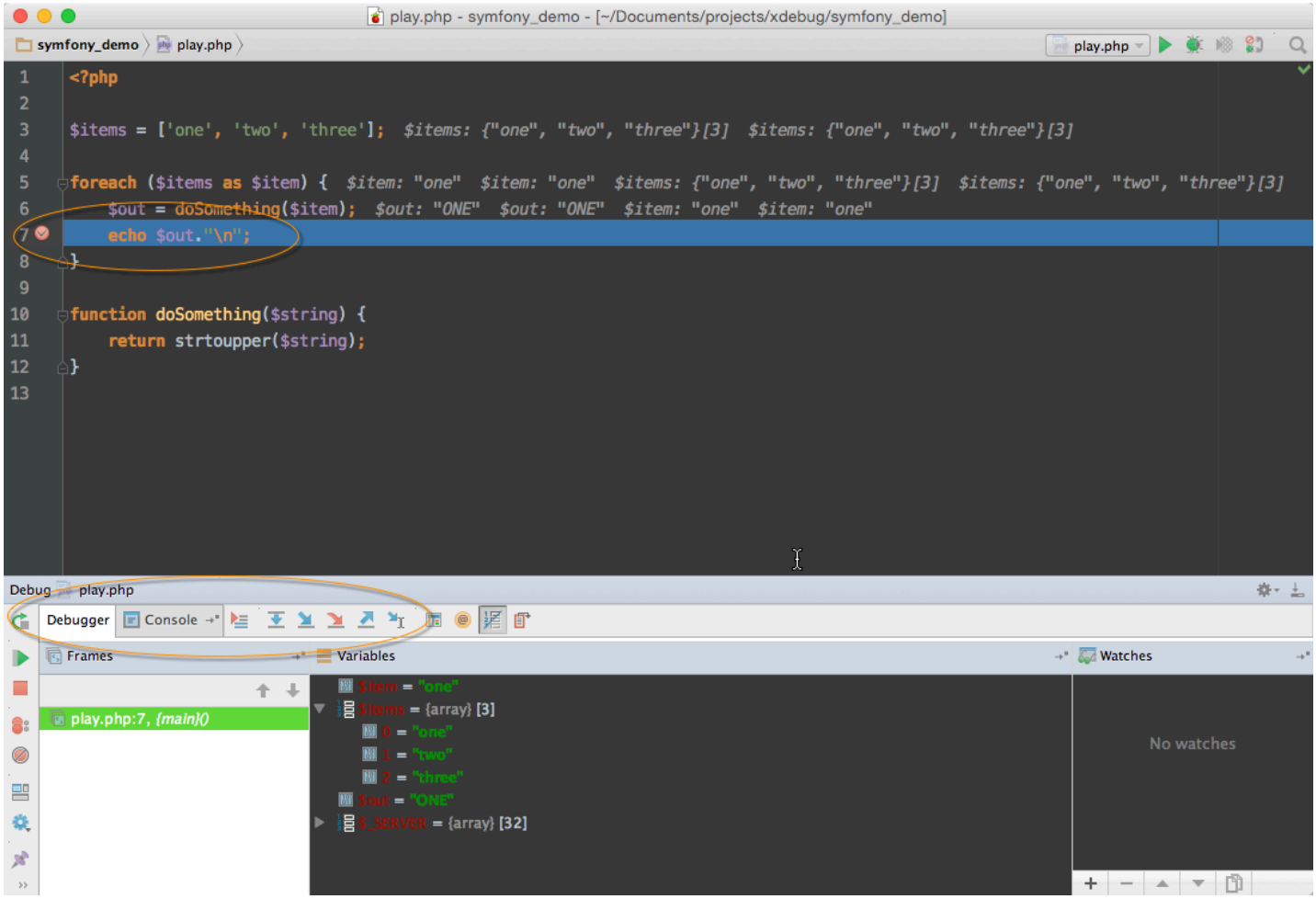


Debug a Script in PhpStorm

The current setup is sufficient for debugging a self-contained PHP script on your local machine. Just set a breakpoint somewhere and start the debugging session with **Run > Debug ...**. The first time, PhpStorm asks you what configuration to use. Pay attention to not accidentally select the JavaScript configuration instead of PHP, because the icons look very similar.



Your selection will by saved automatically as a debug configuration by PhpStorm and remembered for the next time. You are now able to debug your script:



If you've by mistake selected the JavaScript configuration, PhpStorm will open a web browser for the debugging session and you're somehow trapped. You can correct this by going to **Run > Edit configurations ...** and remove the false configuration for your script. Then restart the process and select the correct configuration.

Debug Website on Local Machine

The next level of debugging is to debug a local website. Obviously, some kind of web server must be involved. To keep everthing not directly related to debugging as minimal as possible again, the demo will make use of PHP's internal web server. So there's no 3rd party software involved and we won't get distracted by configuring something not directly related to PHP or Xdebug.

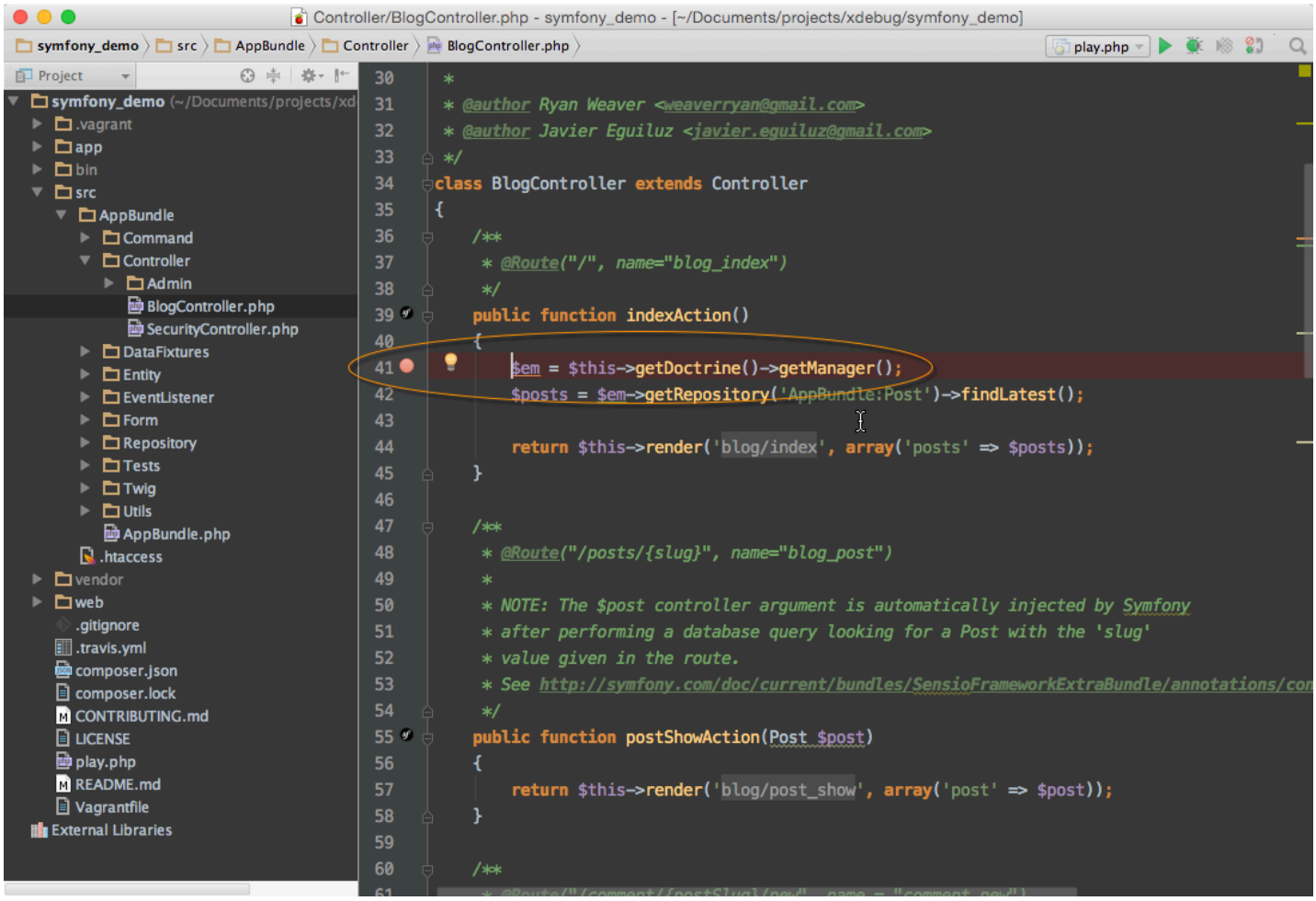
For demonstration purposes I'm using an unmodified Symfony Demo Application. With the Symfony Installer in place, there's nothing more to it than running ...

```
$ symfony demo
$ cd symfony_demo
```

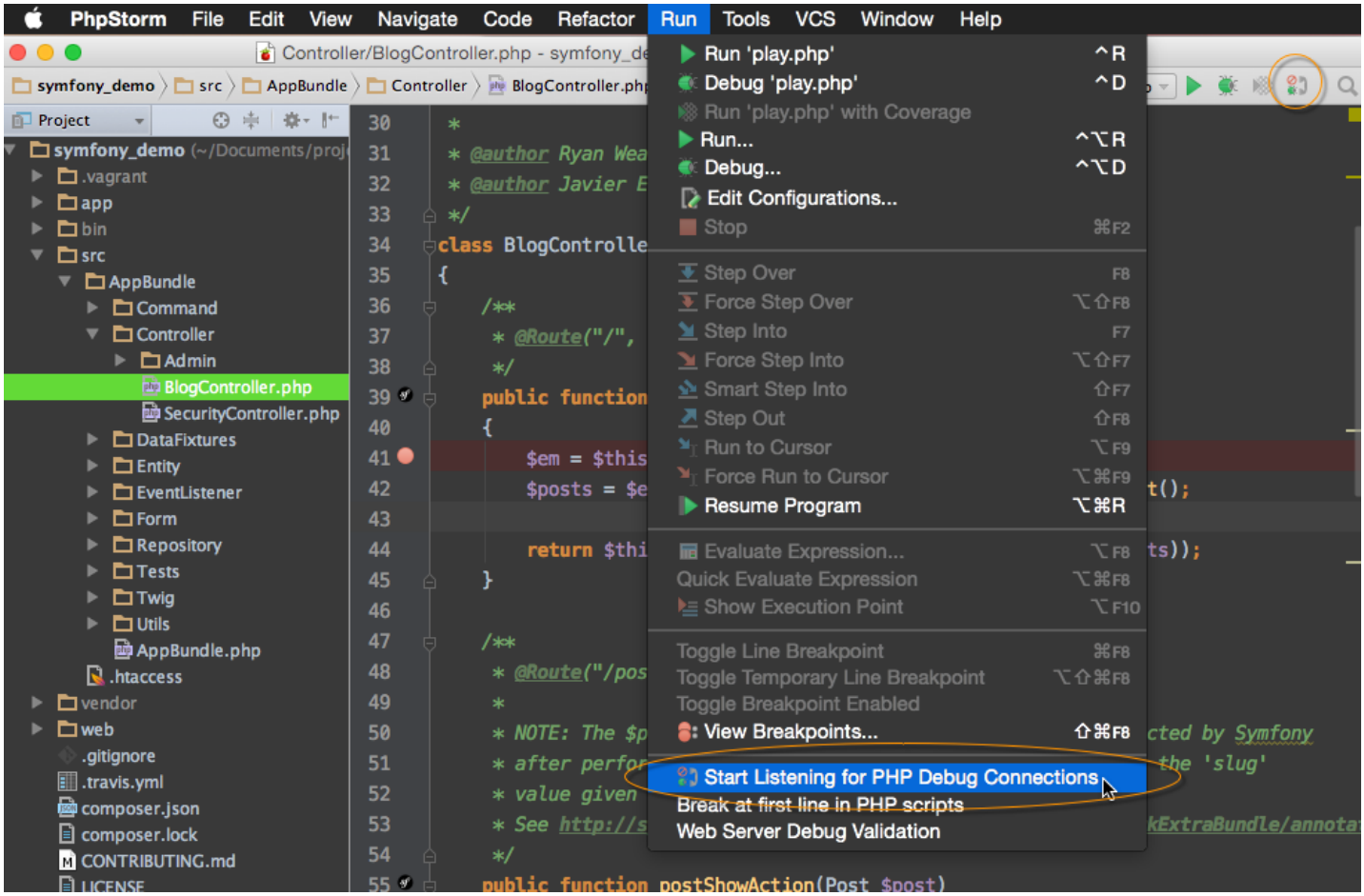
... and opening the new project in PhpStorm. After firing up PHP's internal web server doing ...

```
$ php -S 127.0.0.1:8000 -t web/
```

... you're able to browse the web application at `http://127.0.0.1:8000/app_dev.php`. As the homepage of the Symfony Demo is just a static page without a custom controller, let's click on **Browse application**, which brings us to `http://127.0.0.1:8000/app_dev.php/en/blog/`. The associated controller can be found in the codebase at `AppBundle\Controller\BlogController` and for demo puposes, it seems a good fit to put a breakpoint somewhere in `BlogController::indexAction()`.



Now the only thing you need to do in PhpStorm is to enable listening for incoming debug connections with **Run > Start Listening for PHP Debug Connections** or to click the telephone receiver icon in your toolbar so that it displays green:



You can establish a telnet connection to check out that PhpStorm is now listing on `localhost:9000` (default settings):

```
$ telnet localhost 9000
Trying ::1...
telnet: connect to address ::1: Connection refused
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
```

To quit the telnet session, type `Ctrl+]` followed by `quit`. However, when PhpStorm is not listening for incoming debug connections, no telnet connection can be established:

```
$ telnet localhost 9000
Trying ::1...
telnet: connect to address ::1: Connection refused
Trying 127.0.0.1...
telnet: connect to address 127.0.0.1: Connection refused
telnet: Unable to connect to remote host
```

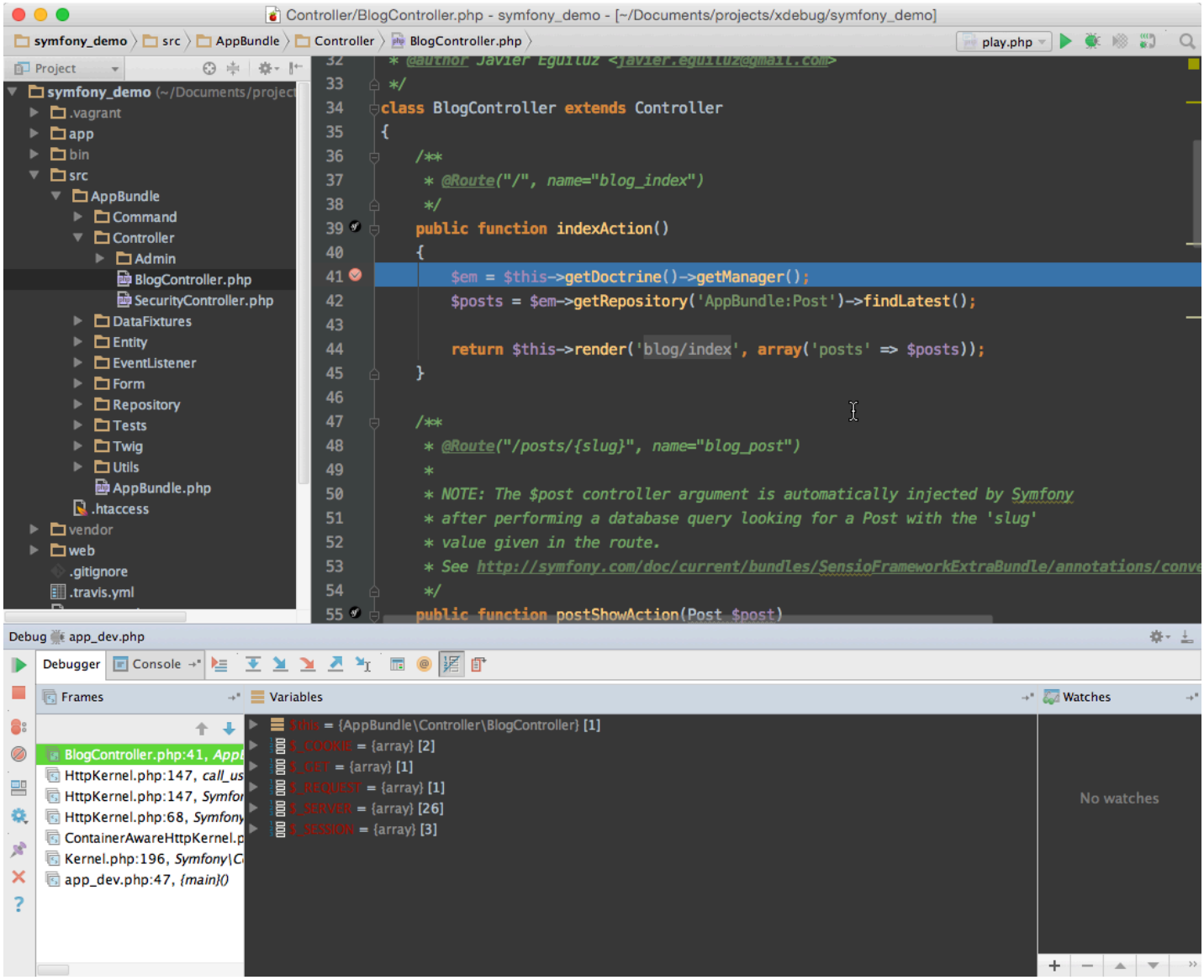
Do not forget to enable the listening again and go back to your browser. Refreshing the page does not yet establish a connection to the debugger because we're missing two things. First, we need to restart the web

server and enable remote debugging in the Xdebug configuration:

```
$ php -d xdebug.remote_enable=1 -S 127.0.0.1:8000 -t web/
```

The `-d` switch of the PHP executable allows for setting php.ini directives that are only valid for the current command. Later, in your development environment, your would of course add this setting to the permanent configuration. But this way you can again see what's really important for which debugging approach – this step was not yet needed in the previous section.

Second, we need to somehow tell our web application that we would like to start a debugging session. This is done by setting a cookie called `XDEBUG_SESSION` with a value representing the so-called *idekey* (IDE key). Xdebug will set this cookie for you when appending the GET parameter `XDEBUG_SESSION_START=some-idekey` to the URL you wish to debug. The actual value of this parameter is not relevant for the so-called "zero configuration" debugging setup in PhpStorm. After navigating to `http://127.0.0.1:8000/app_dev.php/en/blog/?XDEBUG_SESSION_START=foobar`, you can verify that a cookie has been set using the developer tools in the browser of your choice. But even more important, a successful debugger connection should have been established between the browser and PhpStorm, stopping at the configured breakpoint:



With the cookie present, your web application will now try to connect to the debugger for every page request in your application. You can revoke this behavior by simply deleting the cookie either manually or by appending the GET parameter `XDEBUG_SESSION_STOP`: `http://127.0.0.1:8000/app_dev.php/en/blog/?XDEBUG_SESSION_STOP`. Of course, you can also simply stop listening for incoming debug connections in PhpStorm.

Dealing with the cookie that way becomes quite cumbersome after a while. But only now, after knowing what's going on under the hood it's a good idea to switch to a browser extension that does this job on your behalf. For Chrome, there is the Xdebug Helper extension, for other browsers you should easily find the equivalent.



Clicking on **Debug** or **Disable** means just setting and removing the `XDEBUG_SESSION` cookie, respectively. Thus, the extension allows you to easily enable and disable debugging the same way you do it in PhpStorm using the telephone receiver button.

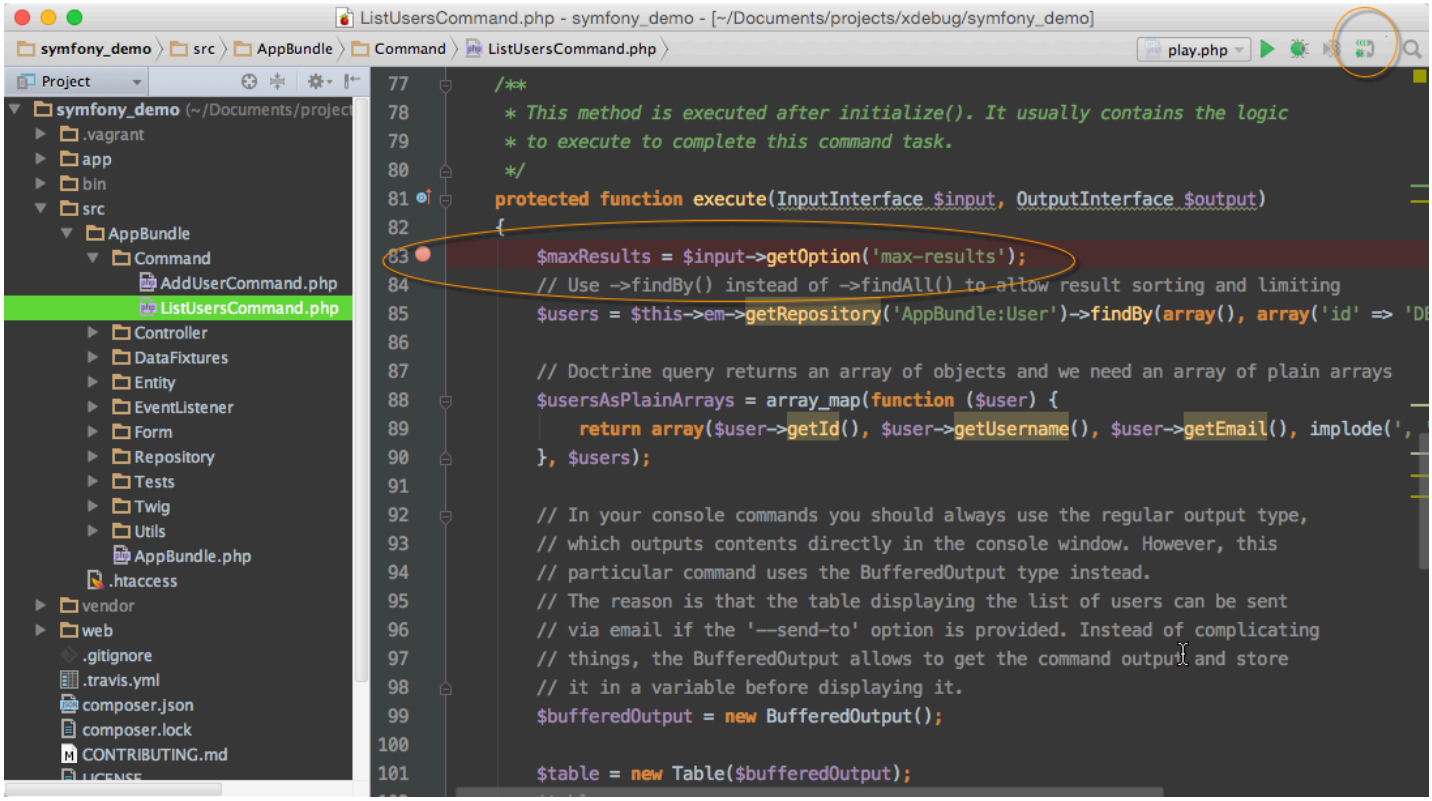
Notice: The debugger connection is working without further configuration because we're sticking to the defaults in Xdebug and PhpStorm: Xdebug tries to connect to `xdebug.remote_host=localhost` on `xdebug.remote_port=9000`. PhpStorm runs on localhost and is listening on port `9000`, see **Preferences > Languages & Frameworks > PHP > Debug** (for PhpStorm 9).

Debug CLI script on Local Machine

For this demo, let's say we want to debug the `app:list-users` command of the Symfony demo application:

```
$ app/console app:list-users
+----+------------+-----------------------+------------+
| ID | Username   | Email                 | Roles      |
+----+------------+-----------------------+------------+
| 2  | anna_admin | anna_admin@symfony.com | ROLE_ADMIN |
| 1  | john_user  | john_user@symfony.com  | ROLE_USER  |
+----+------------+-----------------------+------------+
```

First, ensure that PhpStorm is still listening for a debugging connection, i.e. that the telephone receiver icon is still on green. Second, add a breakpoint somewhere in the `AppBundle\Command\ListUsersCommand::execute()` method, which gets executed for this command.



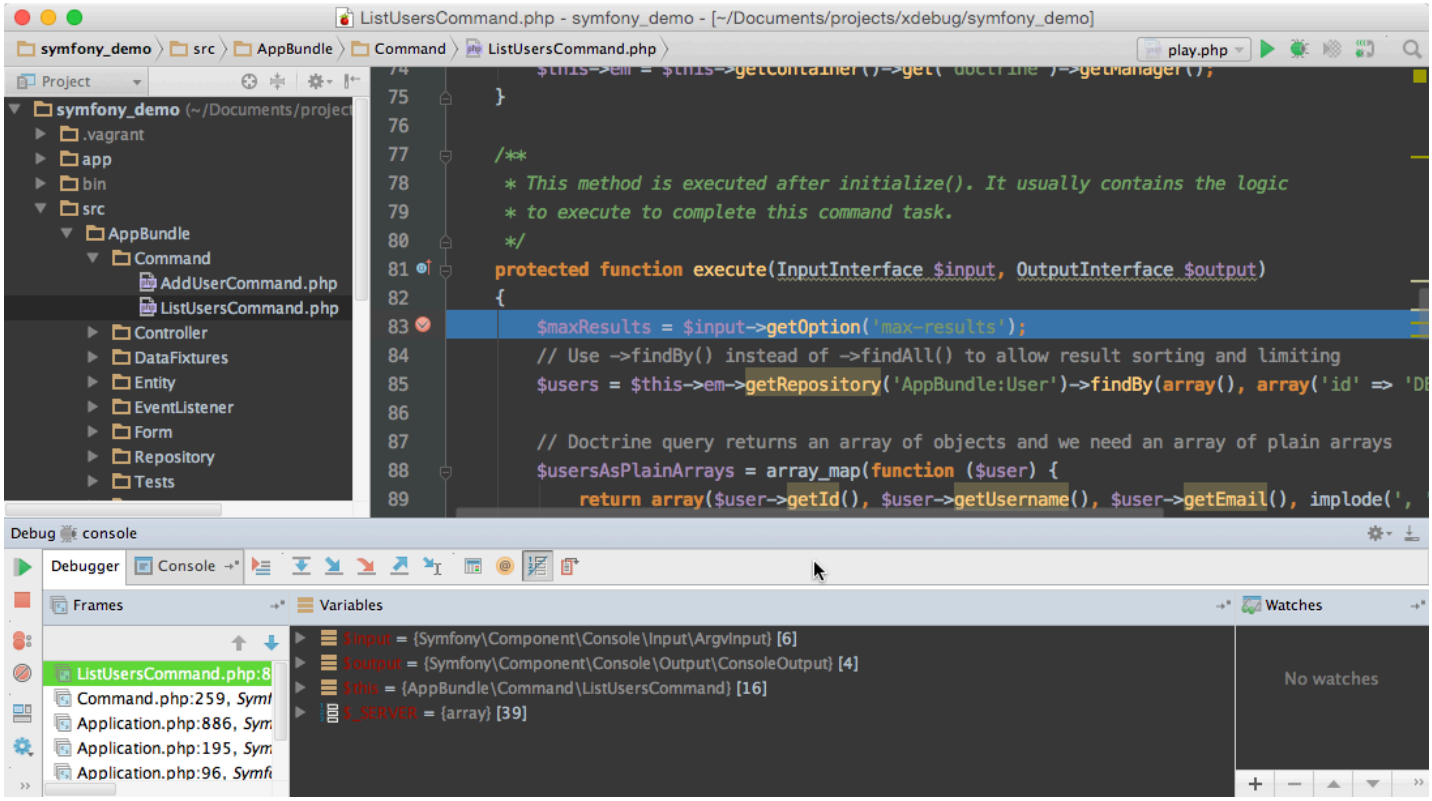
Next, try to execute the command with Xdebug remote debugging enabled:

```
$ php -d xdebug.remote_enable=1 app/console app:list-users
```

Sadly, the debugger does not yet respond to our wish. Connecting to the debugger from the command line is a bit different than from the browser, because we cannot set a cookie to tell PHP that we wish to start a debugging session. Instead, to indicate our intention, an environment variable called `XDEBUG_CONFIG` needs to be set. You can do this for just one single command by simply prepending it to the command:

```
$ XDEBUG_CONFIG= php -d xdebug.remote_enable=1 app/console app:list-users
```

It doesn't matter that it's set to an empty value, Xdebug cann still recognize our intent and PhpStorm will initialize a debugging session:



The value of `XDEBUG_CONFIG` may be used to alter some Xdebug configuration settings. You can add one or more Xdebug directives (space separated), omitting the `xdebug.` prefix:

```
$ XDEBUG_CONFIG="remote_port=9001" php -d xdebug.remote_enable=1 app/console app:list-users
```

The latter command will try to connect to the debugger on port `9001` instead of the default `9000`. If you change the default setting in PhpStorm's **Debug** preferences to the same port, the connection will work again.

Debug Website on Vagrant Machine

A usual setup on a development machine is to have the IDE on the host machine but to put the application into a headless Vagrant virtual machine – providing it's own software stack for the specific project. In regard to debugging, the challenge here is to connect from a different host (the virtual machine) to PhpStorm on the host machine.

Remembering the mantra of only using a minimal configuration to get a working example, this is the demo machine's Vagrantfile:

```
Vagrant.configure(2) do |config|
  config.vm.box = "ubuntu/trusty64"
  config.vm.network :private_network, ip: "192.168.33.99"

  config.vm.provision "shell", inline: <<-SHELL
    sudo apt-get update
    sudo apt-get install -y php5-cli php5-sqlite php5-xdebug
  SHELL
end
```

It creates a fresh Ubuntu LTS installation and assigns the static IP `192.168.33.99` to the virtual machine. During the simple shell provisioning, we only install three packages:

- the CLI package of PHP is just enough to use the built-in web server and to execute CLI commands
- the SQLite driver is needed for the Symfony demo appplication
- PHP's Xdebug extension

Let's boot the virtual machine (doing the provisioning) and SSH into the machine:

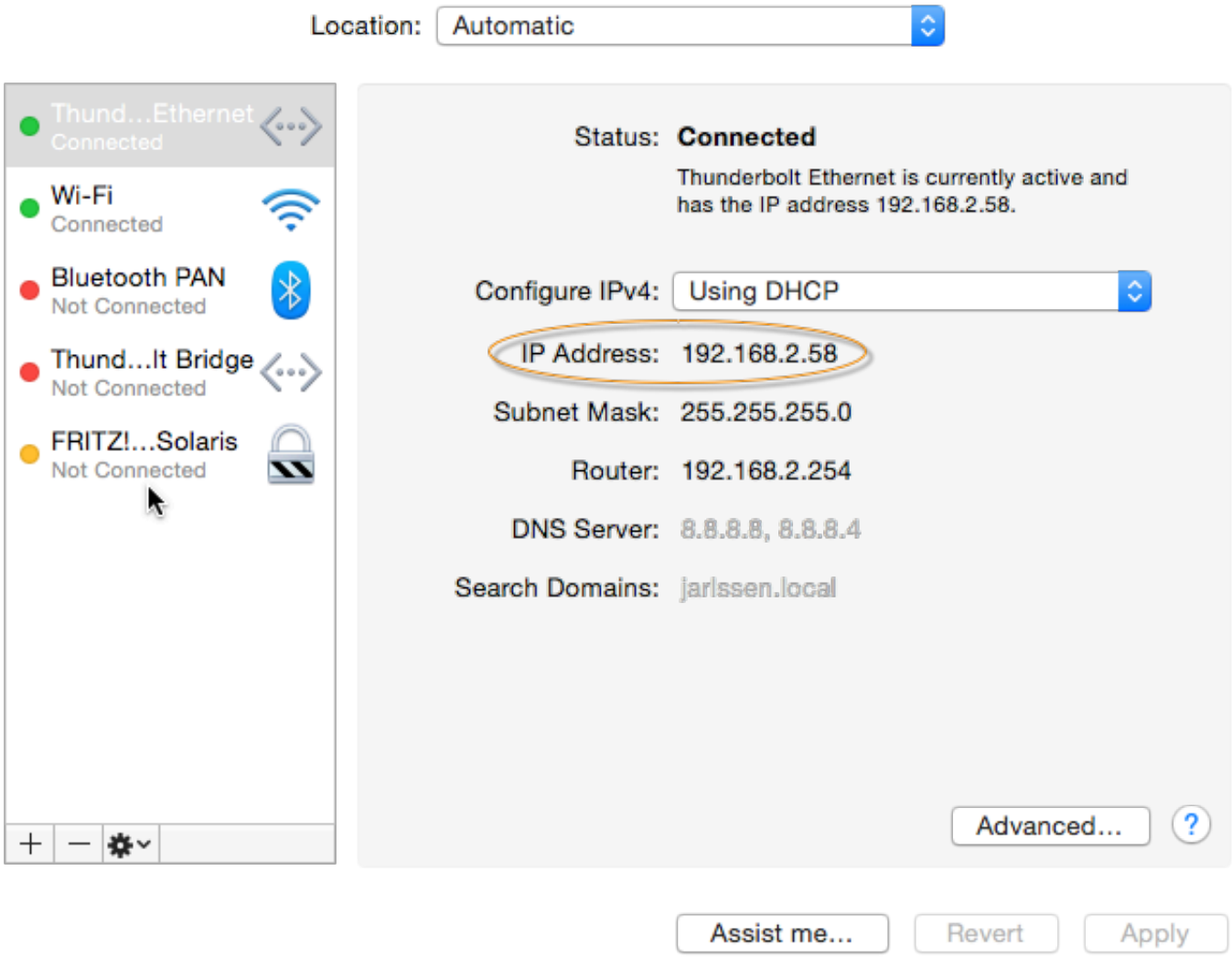
```
$ vagrant up
$ vagrant ssh
$ cd /vagrant
```

Now we can fire up PHP's internal webserver and bind it to the virtual machine's IP address on port `8000`:

```
$ php -S 192.168.33.99:8000 -t web/
```

When navigating to `http://192.168.33.99:8000/app_dev.php`, we are again able to browse the Symfony demo application, but now served from the Vagrant machine. To re-use the existing breakpoint in `AppBundle\Controller\BlogController::indexAction()`, click on **Browse application**, which again brings us to the blog post overview at `http://192.168.33.99:8000/app_dev.php/en/blog/`. Ensure that you're listing to incoming debug connections in PhpStorm by checking the green telephone receiver icon and set your browser into debug mode by activating the `XDEBUG_SESSION` cookie with the browser extension of your choice.

The only missing piece now seems to restart the web server with remote debugging enabled. But wait: how should Xdebug know which machine it should try to connect to? The default setting for `xdebug.remote_host` is `localhost`. But PhpStorm is not running on the Vagrant machine, right? So let's figure out the local IP of the host machine, e.g. on Mac OSX using the `ifconfig` command or by checking **System Preferences > Network > IP Address**.

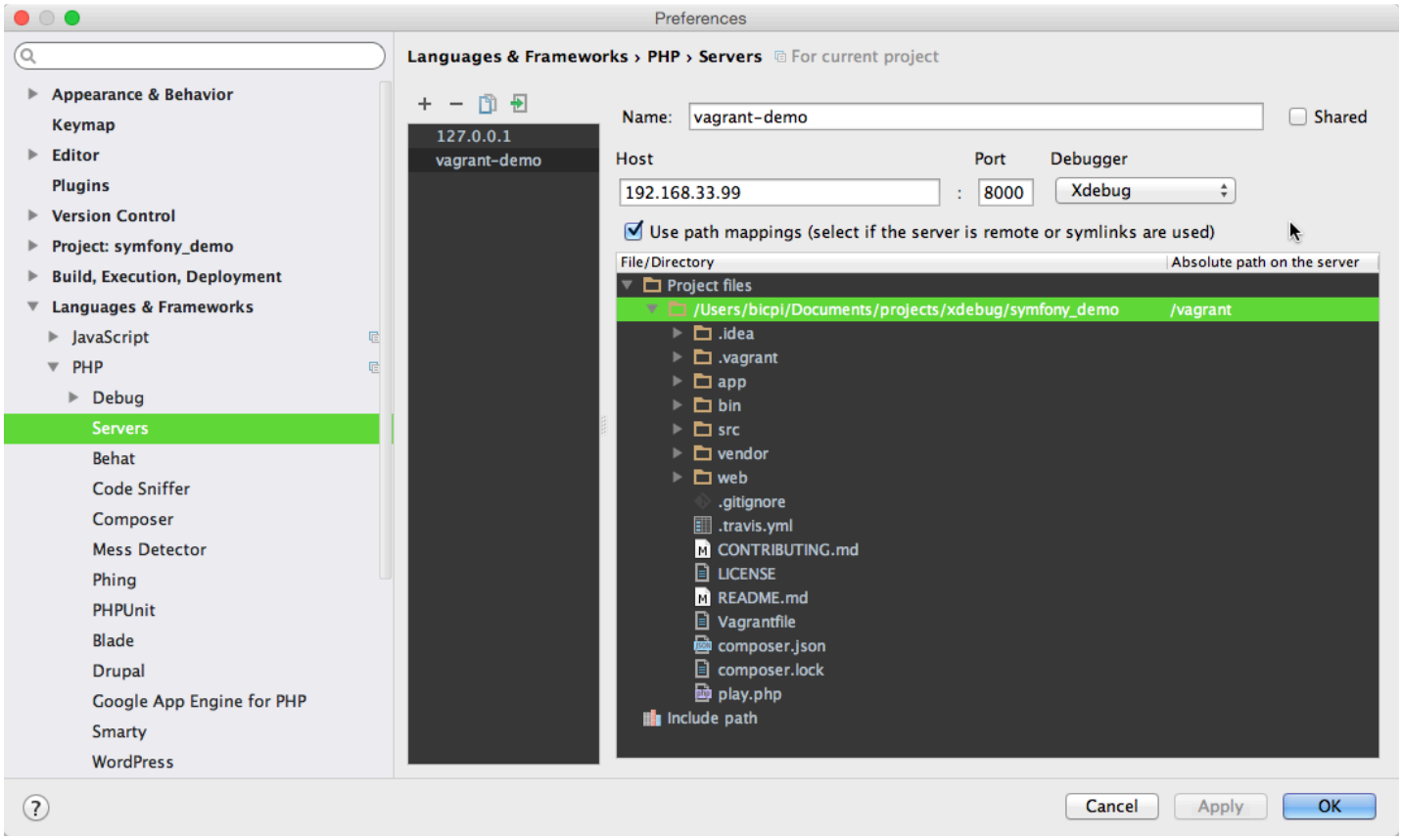


I'll assume `192.168.2.58` for this example and use it for PHP's `xdebug.remote_host` setting, so Xdebug knows where the debugger lives:

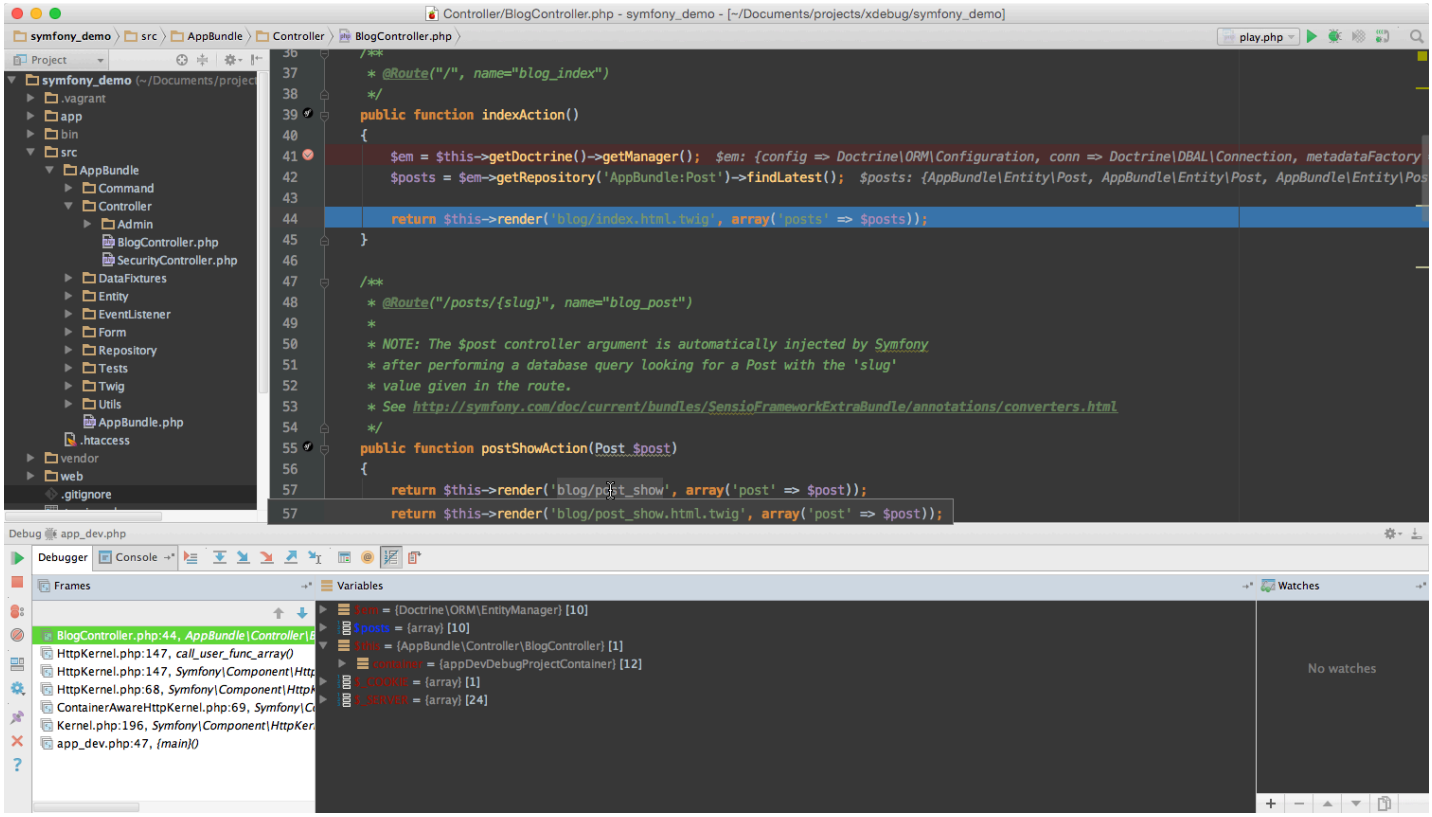
```
$ php -d xdebug.remote_enable=1 -d xdebug.remote_host=192.168.2.58 -S 192.168.33.9
9:8000 -t web/
```

But wait again, how should PhpStorm know which PHP file on the Vagrant machine corresponds to which file on the local machine? For instance, on the local machine, the path to the project's codebase might be `/Users/bicpi/Documents/projects/xdebug/symfony_demo`, but on the Vagrant machine, it's just `/vagrant`. This is why PhpStorm needs a so-called *Path Mapping*. In our Symfony demo application and for this specific Vagrant server, the path mapping needs to indicate that the root path `/vagrant` from the Vagrant machine maps to `/Users/bicpi/Documents/projects/xdebug/symfony_demo` on the local machine. To create a path mapping in PhpStorm, you need to create a *Server* configuration. Servers can be configured in **Preferences > Languages & Frameworks > PHP > Servers** (PhpStorm 9). Add a new server by clicking on the **+** icon above the server list, name it for example `vagrant-demo`, provide the IP address (`192.168.2.58`) and port (`8000`) of the Vagrant machine and configure the above mentioned path

mapping:



When refreshing the webpage for the first time, PhpStorm will now be able to intercept the debug connection and match it against the server configuration deriving it from the IP/hostname and port of the accessed website. Using the path mapping, you'll not notice any difference to debugging a local application:



A little tweak in Xdebug's configuration can disburden us from the need to identify the IP of the host machine. Since we're navigating from the host machine to the remote site with a browser, enabling `xdebug.remote_connect_back` tells Xdebug to simply connect back to the IP where the request came from. So restarting the web server using ...

```
$ php -d xdebug.remote_enable=1 -d xdebug.remote_connect_back=1 -S 192.168.33.99:8
000 -t web/
```

... leads to the same result as before, but no need to specify the local IP of the host machine. This is especially useful in a persistent Xdebug configuration, as it is reusable and distributable independantly from local network conditions.

Debug CLI Script on Vagrant Machine

With all the knowledge from above it should now be as easy as pie to debug a CLI command running on a Vagrant machine. But we have to overcome some obstacles again. First, ensure the previous breakpoint in `AppBundle\Command\ListUsersCommand::execute()` and that PhpStorm is listening for incoming debug connections. Then SSH into the Vagrant machine ...

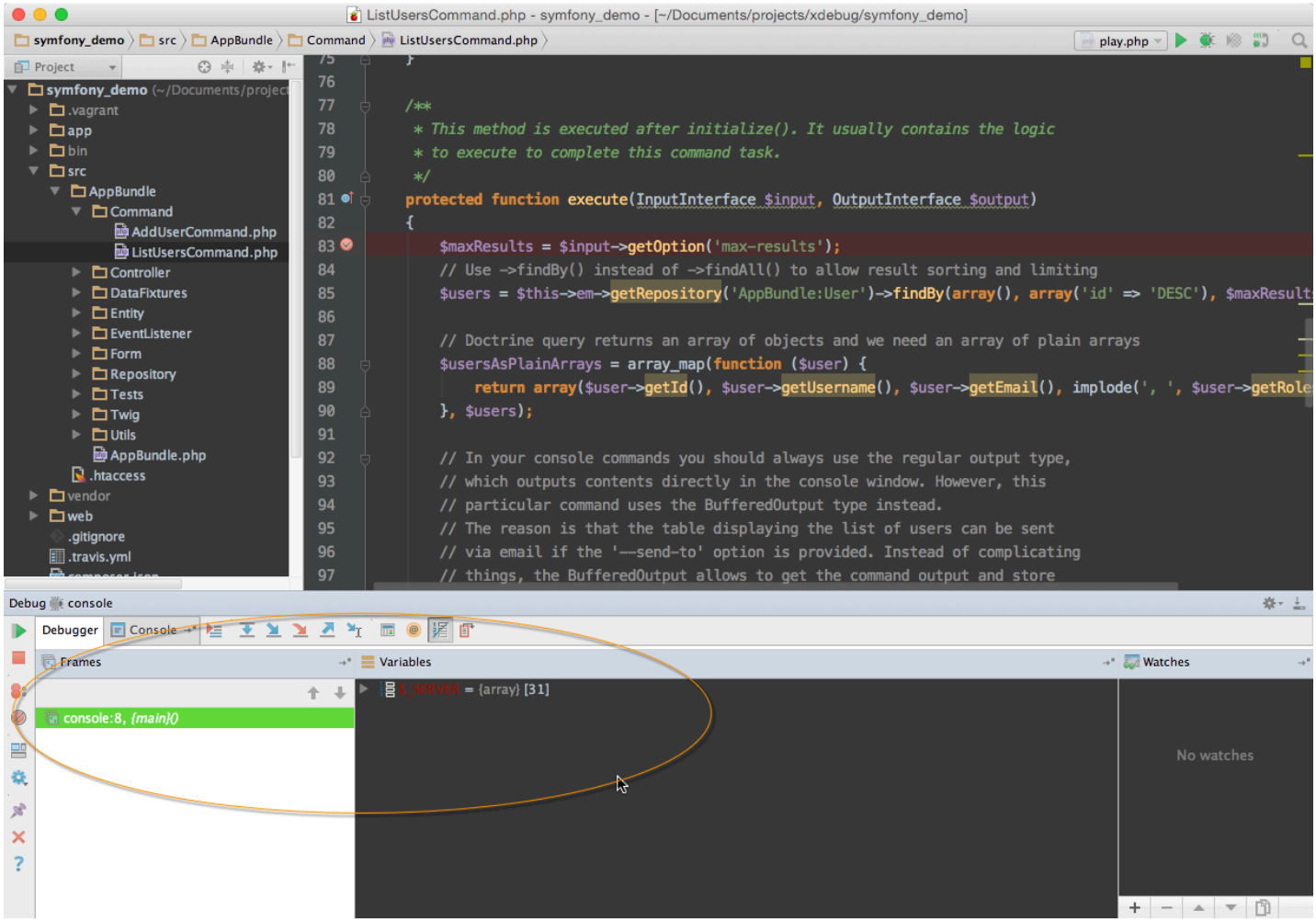
```
$ vagrant ssh
$ cd /vagrant
```

... and verify the command we want to debug:

```
$ app/console app:list-users
+----+------------+-----------------------+------------+
| ID | Username   | Email                 | Roles      |
+----+------------+-----------------------+------------+
| 2  | anna_admin | anna_admin@symfony.com | ROLE_ADMIN |
| 1  | john_user  | john_user@symfony.com  | ROLE_USER  |
+----+------------+-----------------------+------------+
```

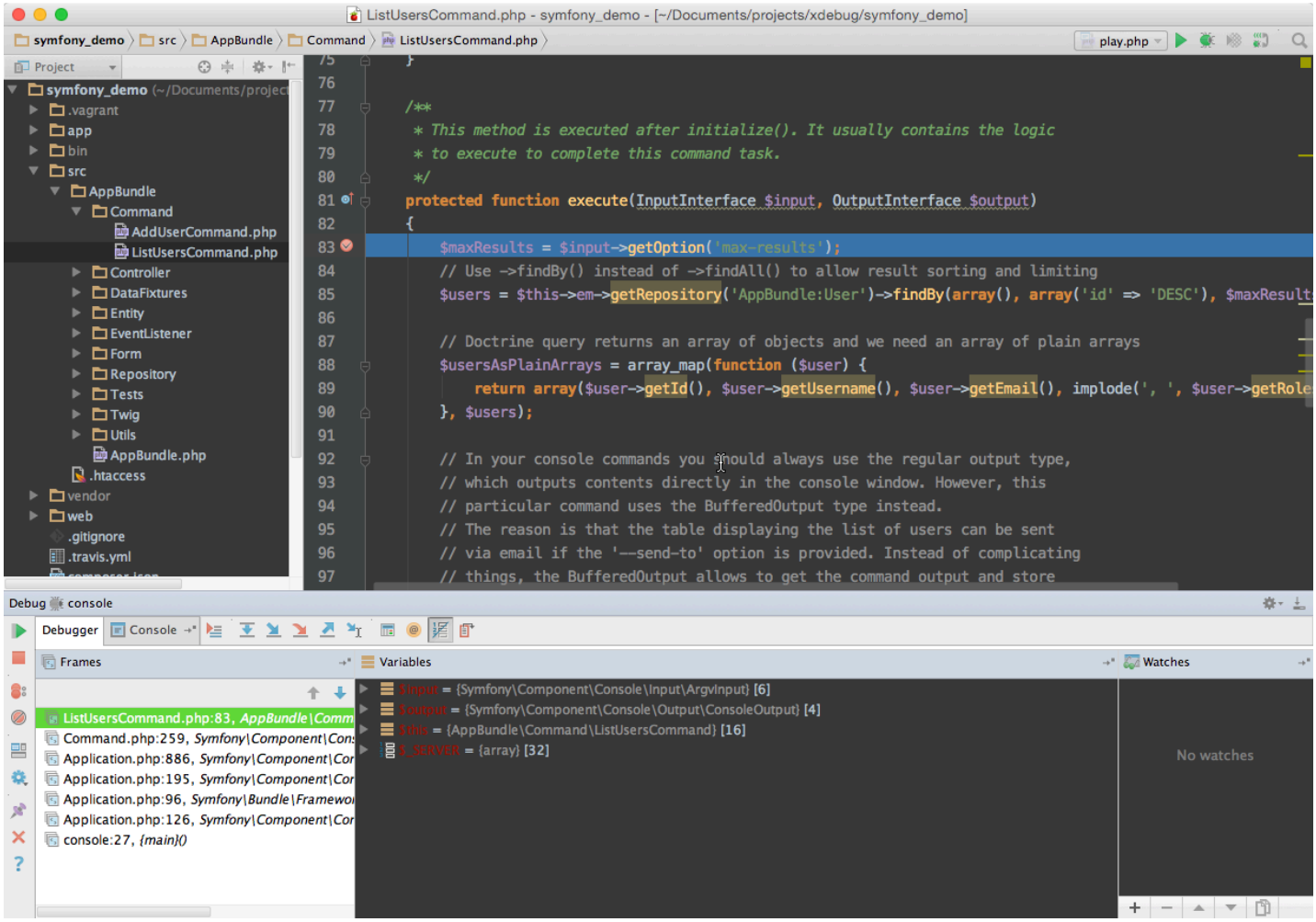
As before on the local machine, we need to declare the debugging intention by setting the `XDEBUG_CONFIG` environment variable. And as there is no browser involved telling Xdebug the target IP of the debugger, we need to do this manually using the `xdebug.remote_host` directive again:

```
$ XDEBUG_CONFIG= php -d xdebug.remote_enable=1 -d xdebug.remote_host=192.168.2.58
app/console app:list-users
```



The connection was established successfully, but the **Frames** and **Variables** sections look somehow incomplete. So what's wrong here? Well, remember the path mapping from the last section? How should PhpStorm know which server configuration to use? For the web application it could be derived from the URL/IP we accessed with the browser from the host machine. But for the CLI it's the other way round, we're initializing the debug session on the remote Vagrant machine. So we somehow need to pass an information to Xdebug which server configuration PhpStorm should use. Another environment variable comes to the rescue: `PHP_IDE_CONFIG`. The value of this variable let's you specify the server name of the appropriate configuration with the key `serverName` (this is case sensitive!) and the path mapping is correct now:

```
$ PHP_IDE_CONFIG="serverName=vagrant-demo" XDEBUG_CONFIG= php -d xdebug.remote_ena
ble=1 -d xdebug.remote_host=192.168.2.58 app/console app:list-users
```

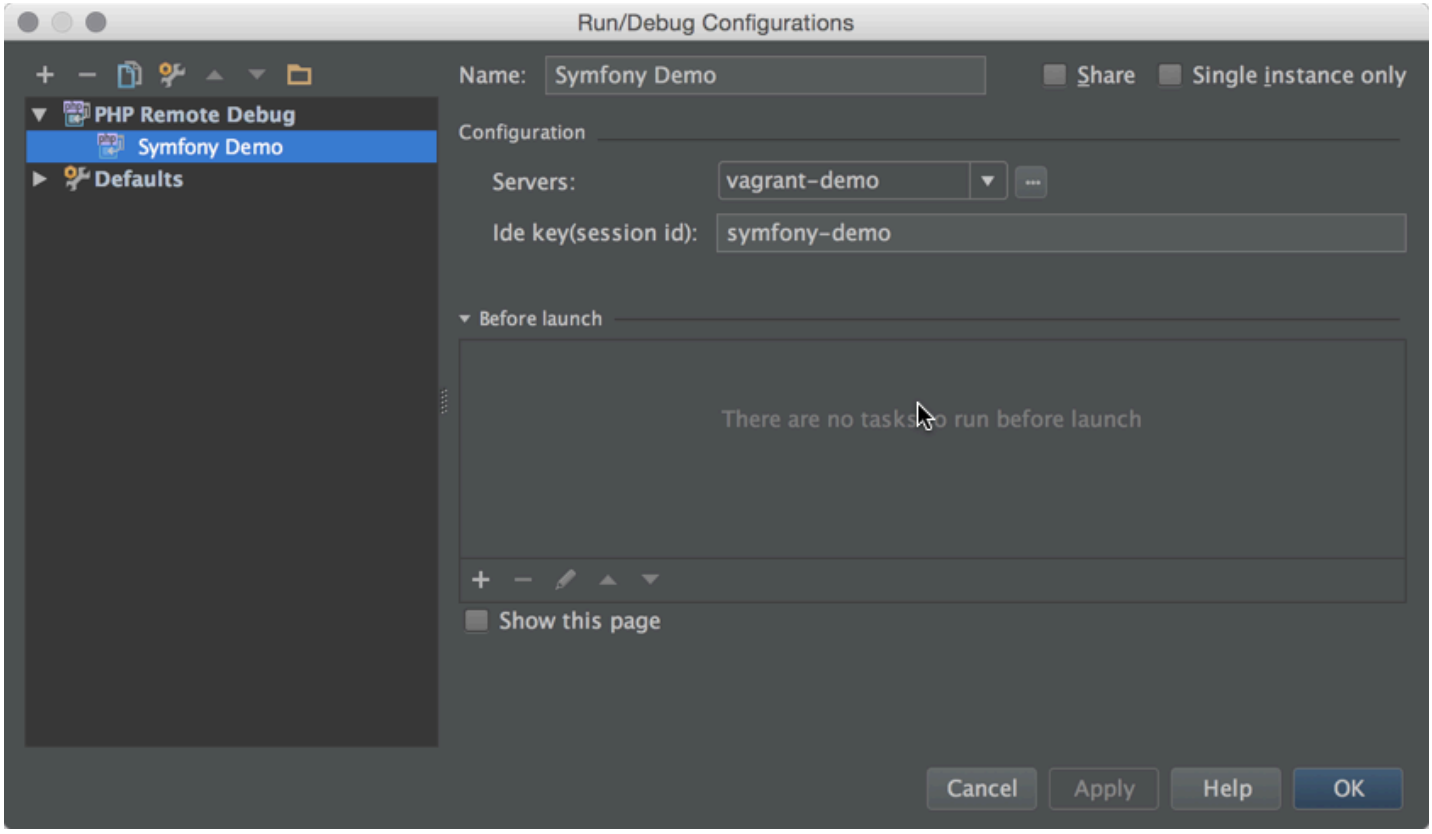


Using PhpStorm Configurations

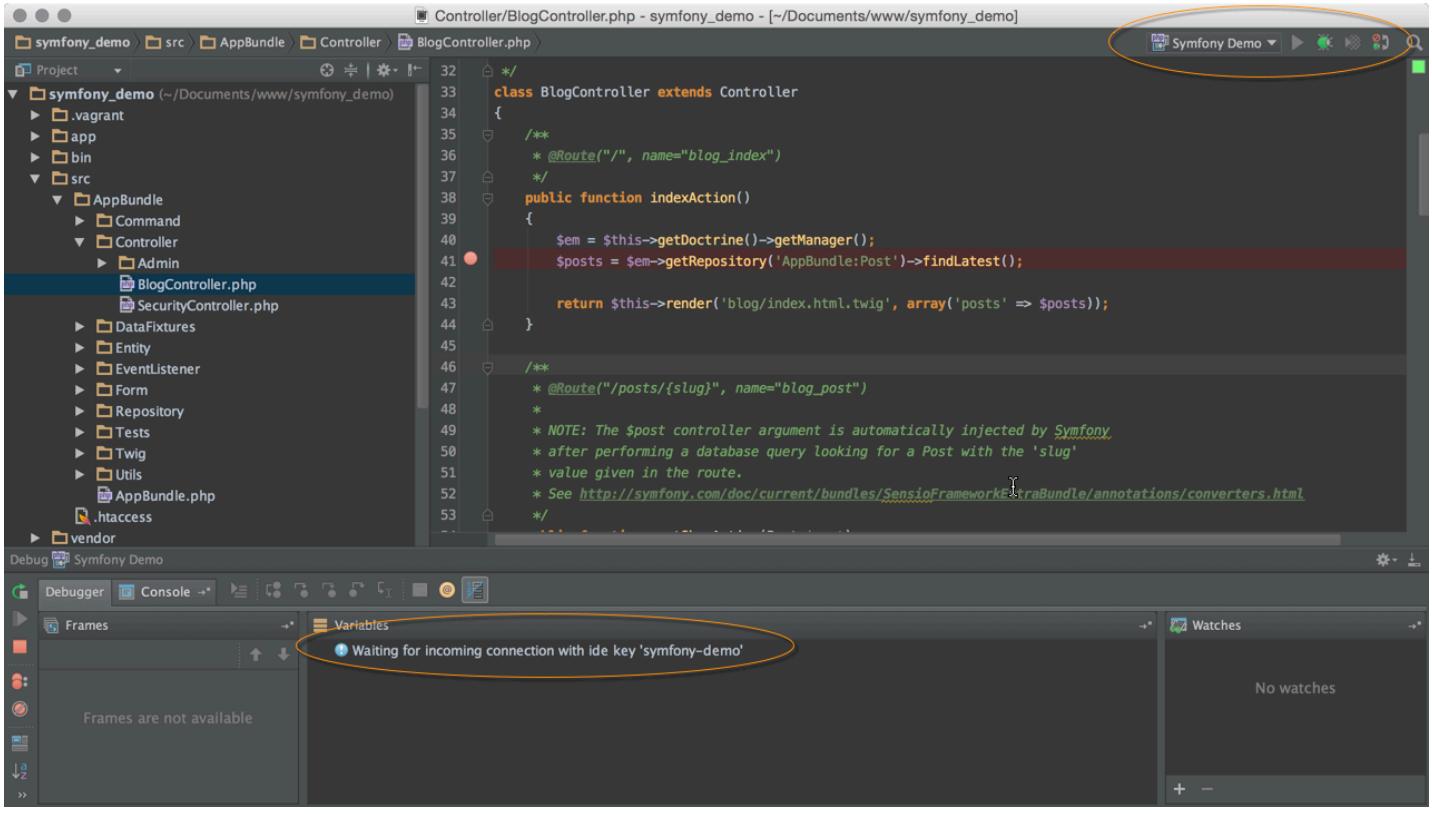
Even if the previous sections should cover everything you'll need for successfully setup debugging connections, PhpStorm's debug configurations feature might come in handy to simplify you day-to-day work. For instance, it allows you to start debugging a web application with a single click or supports you in special situations, e.g. if you need to run an external program before each debugging session etc. Even if you never make use of the debug configurations, you can still learn something by studying how they work.

PHP Remote Debug

To setup PHP remote debugging, go to **Run > Edit Configurations...**, click on the `+` icon (*Add New Configuration*) above the list of configurations on the left and select **PHP Remote Debug**. Enter a name for the new configuration, assign the related server (we'll reuse the Vagrant server configuration from above) and an IDE key of your choice (we'll use `symfony-demo`). The *Before launch* section allows you to hook into the launch process executing further tasks.



Click **OK** and the **Run > Debug...** to start a debugging session and select the newly created debug configuration from the popup that appears. Alternatively, you can also select the configuration from a dropdown in one of the toolbars and click on the *Debug* icon. PhpStorm will open the debug console and let us know that it is waiting for an incoming debug connection with the IDE key *symfony-demo*. The status of the telephone receiver icon is not relevant for this kind of connection.



But how to initialize the debugging session now? Well, we just need to initialize it from the browser or the console unsing the appropriate IDE key. As you might remember from above, this can be done in the web browser by setting the `XDEBUG_SESSION` cookie to the appropriate value. For this example, it means to include the GET parameter `XDEBUG_SESSION_START=symfony-demo` in the URL to the page we wish to debug, e.g. `http://127.0.0.1:8000/app_dev.php/en/blog/?XDEBUG_SESSION_START=symfony-demo`. If you're using a browser extension, there should also be a way to configure the value if the IDE key. In Chrome's *Xdebug Helper*, you can right click on the icon in the address bar, go to *Options* and then change the value.

On the CLI, however, the IDE key can be specified through the `XDEBUG_CONFIG` environment variable (no need for `PHP_IDE_CONFIG` any more as the server configuration can be derived from the debug configuration):

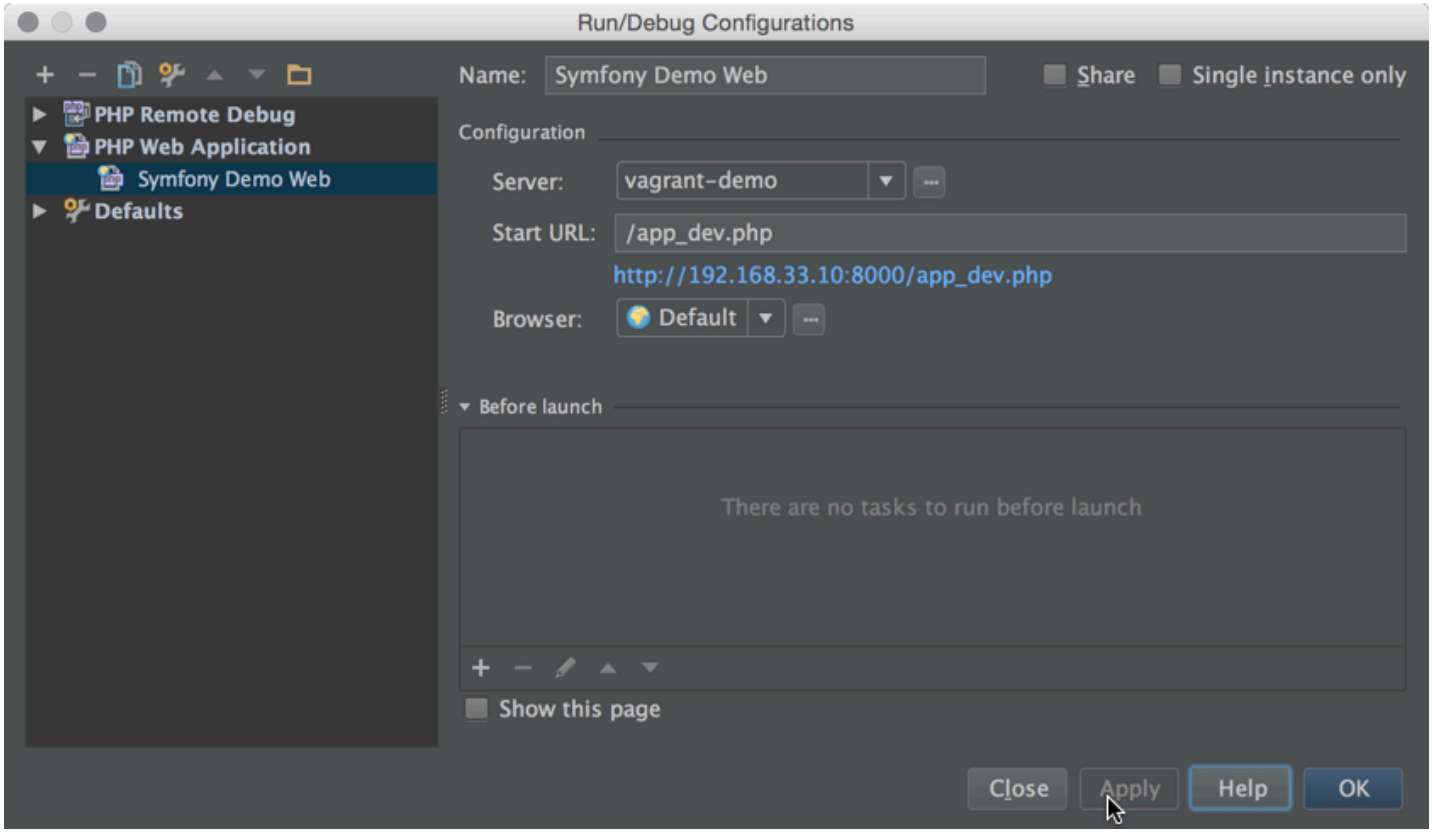
```
$ XDEBUG_CONFIG="idekey=symfony-demo" php -d xdebug.remote_enable=1 -d xdebug.remo
te_host=192.168.2.58 app/console app:list-users
```

To sum it up: Using a Remote Debug Configuration, PhpStorm is just listing for a debug connection with a precise IDE key. It also allows you to customize the debugging session.

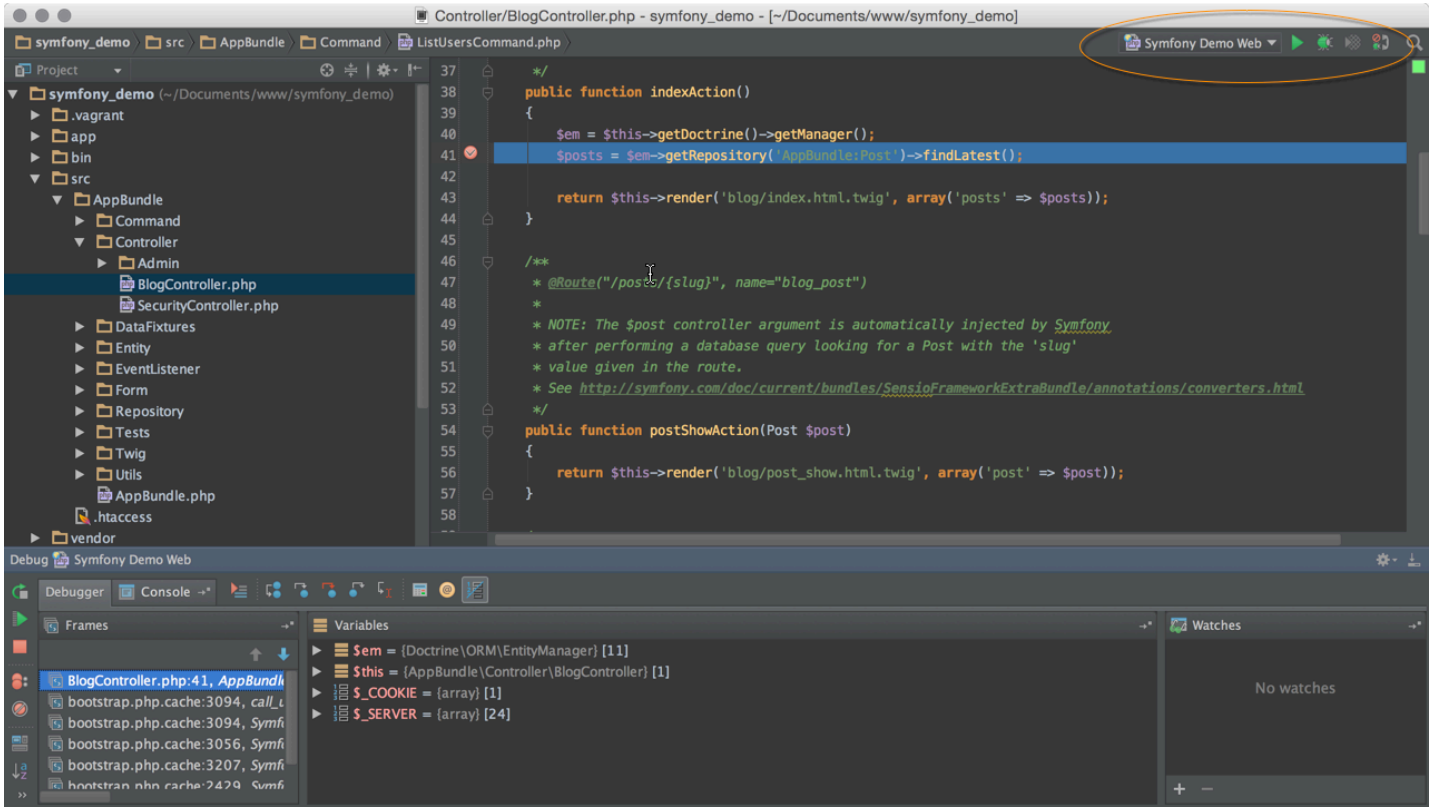
PHP Web Application

You can also setup a "One-Click" solution to debug a web application. One click on the debug icon opens the browser and initializes a debugging session, i.e. no need to struggle with an IDE key, configure the browser or prepare PhpStorm for incoming connections.

To setup PHP Web Application debugging, go to **Run > Edit Configurations...**, click on the `+` icon (*Add New Configuration*) above the list of configurations on the left and select **PHP Web Application**. Enter a name for the new configuration, assign the related server (we'll again reuse the Vagrant server configuration from above) and an initital relative start URL (we'll use `/app_dev.php`). The *Before launch* section allows you to hook into the launch process executing further tasks.



A debugging session can be initialized just the same way we've seen in the previous session. But this time, a click on the debug icon opens the browser, sets the `XDEBUG_SESSION` cookie to a random IDE key (watch for it in the URL) and PhpStorm starts listening for incoming debug requests for this IDE key.



Please note that Xdebug still needs to be configured so that it can correctly connect back to PhpStorm on

the host machine, i.e. using `xdebug.remote_host` or `xdebug.remote_connect_back`.

About the author

Philipp Rieber is a certified PHP and Symfony developer working at Session Digital in Munich, Germany. He is also engaged in the frontend, the Cloud, on Mobile, as a DevOp and a technical writer. You can meet him online @bicpi or find some more infos at philipp-rieber.net. Offline, he is never missing the PHP User Group Munich and he is co-organizing the Symfony User Group Munich.